

UNIVERSITÉ SORBONNE PARIS NORD

IUT DE VILLETANEUSE

Département Informatique

SAÉ S2.04 – Exploitation d'une base de données

RAPPORT DE SYNTHÈSE :

Groupe MARS - SQLions de l'Atlas

Hamza EL BOUBAKRI

Anass FATHI

Wassil Seghirouali

Destinataire : M.BRINGUIER

Sous la direction de : M.BRINGUIER

Date de rendu : 31/05/2026

Table des Matières

I – Script manuel de création de la base de données	3page
A - scripts SQL de création des tables.....	3 page
B - les clés primaires et les clés étrangères.....	4 page
C - les contraintes d'intégrité	5 page
II – Peuplement des tables	6 page
A - les scripts d'importation ou d'alimentation des données.....	6 page
III – Justification des choix	7 page
A - explication du mécanisme d'historisation des prix	7 page
B - interprétation des résultats SQL.....	8 page

I – Script manuel de création de la base de données

A - Scripts SQL de création des tables

```
DROP TABLE IF EXISTS rupture ;
DROP TABLE IF EXISTS prix ;
DROP TABLE IF EXISTS posseder;
DROP TABLE IF EXISTS ouverture_horaire;
DROP TABLE IF EXISTS service;
DROP TABLE IF EXISTS carburant;
DROP TABLE IF EXISTS station;
DROP TABLE IF EXISTS departement;

CREATE TABLE departement (
    code_departement VARCHAR(3) PRIMARY KEY,
);

CREATE TABLE d(
    id_station INT PRIMARY KEY,
    pop CHAR(1),
    code_postal VARCHAR(10) NOT NULL,
    nom_ville VARCHAR(250),
    adresse VARCHAR(250),
    latitude NUMERIC,
    longitude NUMERIC,
    code_departement VARCHAR(3),
    FOREIGN KEY (code_departement) REFERENCES departement(code_departement)
);

CREATE TABLE carburant (
    id_carburant INT PRIMARY KEY,
    nom_carburant VARCHAR(50) NOT NULL
);

CREATE TABLE service (
    id_service INTEGER PRIMARY KEY GENERATED ALWAYS AS IDENTITY,
    service_propose VARCHAR(250) NOT NULL
);

CREATE TABLE ouverture_horaire (
    jour VARCHAR(20) NOT NULL,
    id_station INT NOT NULL,
    heure_ouverture TIME,
    heure_fermeture TIME,
```

```
    automate BOOLEAN,  
    PRIMARY KEY (jour, id_station),  
    FOREIGN KEY (id_station) REFERENCES station(id_station)  
);  
  
CREATE TABLE posseder (  
    id_service INT NOT NULL,  
    id_station INT NOT NULL,  
    PRIMARY KEY (id_service, id_station),  
    FOREIGN KEY (id_service) REFERENCES service(id_service),  
    FOREIGN KEY (id_station) REFERENCES station(id_station)  
);  
  
CREATE TABLE prix (  
    id_carburant INT NOT NULL,  
    id_station INT NOT NULL,  
    date_prix_maj TIMESTAMP NOT NULL,  
    prix_carburant NUMERIC(4,3) NOT NULL,  
    PRIMARY KEY (id_carburant, id_station, date_prix_maj),  
    FOREIGN KEY (id_carburant) REFERENCES carburant(id_carburant),  
    FOREIGN KEY (id_station) REFERENCES station(id_station)  
);  
  
CREATE TABLE rupture (  
    id_carburant INT NOT NULL,  
    id_station INT NOT NULL,  
    debut_rupture TIMESTAMP NOT NULL,  
    type_rupture VARCHAR(100) NOT NULL,  
    PRIMARY KEY (id_carburant, id_station, debut_rupture),  
    FOREIGN KEY (id_carburant) REFERENCES carburant(id_carburant),  
    FOREIGN KEY (id_station) REFERENCES station(id_station)  
);  
  
INSERT INTO carburant (id_carburant, nom_carburant)  
VALUES (1,'Gazole'), (2,'SP95'), (3,'E85'), (4,'GPLc'), (5,'E10'), (6,'SP98');
```

B - Clés primaires et les clés étrangères

À partir de notre architecture de base de données, il nous a fallu élaborer deux principaux concepts, qui nous servent d'une part d'avoir des interactions entre les différentes tables de notre modèle de données mais aussi d'autre part de suivre les préceptes fondamentaux d'une donnée, à savoir l'unicité et la non-redondance d'une entité. Pour cela les clés primaires sont là pour encadrer les règles de nos entités, sans oublier les clés étrangères

qui, elles, remplissent un objectif différent . Lier les différentes tables entre elles tout en restant une ressource capitale au bon fonctionnement de notre architecture de données.

Table departement :

Clé Primaire : code_departement

Table station :

Clé Primaire : id_station

Clé Étrangère : code_departement

Table carburant :

Clé Primaire : id_carburant

Table service :

Clé Primaire : id_service

Table ouverture_horaire :

Clé Primaire couple : (jour, id_station)

Clé Étrangère : id_station

Table posseder :

Clé Primaire couple : (id_service , id_station)

Clés Étrangères : id_service , id_station

Table prix :

Clé Primaire triple :(id_carburant , id_station , date_prix_maj)

Clés Étrangères : id_carburant , id_station

Table rupture :

Clé Primaire triple : (id_carburant , id_station , debut_rupture)

Clés Étrangères : id_carburant , id_station

C - contraintes d'intégrité

Les contraintes d'intégrité sont au cœur même des règles de notre architecture de données. En effet, ce sont elles qui définissent les mesures de sécurité pour pouvoir avoir des données qui respectent nos conditions prédéfinies. Par ailleurs, lors de l'importation, ce sont elles qui vont pouvoir donner accès ou stopper les données qui ne respectent pas le format attendu par nos contraintes d'intégrité. Les contraintes d'intégrité prennent plusieurs formes :

La première contrainte est celle définissant les types de valeurs attendu , exemple : integer ,varchar , text ,numeric .Celle ci est doublement contrainte par l'apparition de contrainte au

seins d'elle même pour notamment délimiter la taille possible que peut prendre une donnée , varchar(200) pour 200 caractère possible au maximum.

La seconde contrainte d'intégrité forte se prénomme NOT NULL : celle-ci permet de prédéfinir une donnée comme non-vide, elle est cruciale notamment pour avoir des données capitales telles que id_carburant ou encore son nom qui ne peut être vide pour une architecture de donnée basée sur celle-ci.

Notre troisième type de contrainte a été détaillé plus tôt puisque les clés primaire et étrangère sont des contraintes d'intégrité à part entière faisant partie de notre architecture de données.

Enfin la contrainte d'intégrité GENERATED ALWAYS AS IDENTITY définie pour id_service de l'entité service est une contrainte d'identité beaucoup utilisée lors de notre première SAE. Celle-ci permet à PostgreSQL de pouvoir calculer en temps réel l'identifiant et les attribuer à celle-ci pour celle manquante, ainsi nous évitons de problème en cascade causé par un manque d'id lors de nos différentes importations et commande SQL.

II – Peuplement des tables

A - les scripts d'importation ou d'alimentation des données

Voir principalement le fichier mais explications intéressantes ci dessous

On a fait un script Python pour importer les données. Pour traiter le fichier xml mis en Open Data par le gouvernement (ministère économique et finance). Nous avons utilisé la bibliothèque xml.etree.ElementTree afin de parcourir le fichier XML, et aussi le module psycopg2 pour la connexion à notre environnement PostgreSQL.

Notre script nettoie d'abord les données avec par exemple la division des coordonnées géographiques par 100 000 pour obtenir des latitudes , longitudes standards (dit sur le site), reformatage des heures (remplacement des points par des “.” ,on généraliser tous nos données pour pouvoir avoir des visualisation précise plus tard), et traduction des valeurs booléennes pour les automates.

Pour respecter nos contraintes d'intégrité (clés étrangères, primaire ...) nous faisons des vérifications. Par exemple, on connaît le code_departement grâce aux deux premiers caractères du code postal de la station. On vérifie avec un SELECT si ce département existe déjà dans la base. Si c'est pas le cas, on fait un insert, on le crée tout simplement.

Enfin, lors de l'importation des données depuis le fichier xml celui ci étant assez lourd il a fallu subdiviser cette masse de donnée en plusieurs paquets afin de garder une fluidité et ne pas faire planter notre implémentation due à une surcharge RAM . On faisait l'importation par paquet de 100 stations.

Note importante: on fait l'importation à partir du xml datant du 26 mai 2026, xml de ce jour dans le zip. Ça marche pour les autres jours du mois. Également pour les xml annuel, sauf que c'est trop lourd pour l'ordi (près d'une heure et demi d'exportation pour l'import des 30 derniers jours).

Physiquement c'est dur, à voir si les ordinateurs de l'iut sont beaucoup plus puissants. On compte essayer, on a espoir en ces ordi.

III – Justification des choix

A - explication du mécanisme d'historisation des prix

Le plus gros problème de cette extraction vers notre base de données réside dans le fait que ce soit du flux instantané. Ça montre l'état du marché sur les prix à un moment donné. Si on avait simplement un prix attaché au carburant d'une même station, à chaque fois que nous aurions importé le prix ça aurait écrasé le prix précédent. Et donc, pour pouvoir répondre à ce problème d'historisation des prix et nous permettre de faire des analyses prochainement via grafana voici ce qu'on a fait.

Tout d'abord, on a pu créer une table prix entre carburant et station grâce à la cardinalité n n. Ça nous a permis ensuite d'avoir un triplet une clé primaire à 3 attributs. Comme ça nous pouvions avoir beaucoup de prix pour une même station et pour un même carburant et tout ça dans le temps. C'est la même chose pour rupture.

Ensuite, concernant le script de Python pour l'importation des données nous avons à chaque fois un SELECT pour vérifier que si les prix étaient les mêmes on passait. Sinon on faisait un insert.

B - interprétation des résultats SQL

1- Cette requête nous permet d'avoir une vision vraiment générale sur l'ensemble des stations avec la ville et aussi le carburant qui est vendu. Il y a l'air d'avoir 10 stations par ville de vue.

```
SELECT DISTINCT
  s.nom_ville,
  c.nom_carburant
FROM station s
JOIN prix p ON s.id_station = p.id_station
JOIN carburant c ON p.id_carburant = c.id_carburant
ORDER BY s.nom_ville, c.nom_carburant ASC
LIMIT 67;
```

nom_ville	nom_carburant
abbeville	E10
abbeville	Gazole
abbeville	SP98
Abbeville	E10
Abbeville	E85
Abbeville	Gazole
Abbeville	SP98
ABBEVILLE	E10
ABBEVILLE	E85
ABBEVILLE	Gazole
ABBEVILLE	GPLC
ABBEVILLE	SP95
ABBEVILLE	SP98
Ablis	Gazole
Ablis	SP95
Ablis	SP98
Abondance	Gazole
Abondance	SP95
Abondance	SP98
Abscon	E10
Abscon	Gazole
Abscon	SP98
Accous	Gazole
Accous	SP95
Accous	SP98
ACHERES	E10
ACHERES	E85
ACHERES	Gazole
ACHERES	GPLC
ACHERES	SP98
Achères-la-Forêt	E10
Achères-la-Forêt	E85
Achères-la-Forêt	Gazole
Achères-la-Forêt	GPLC
Achères-la-Forêt	SP98
ACHEUX-EN-AMIÉNOIS	E10
ACHEUX-EN-AMIÉNOIS	E85
ACHEUX-EN-AMIÉNOIS	Gazole
ACHEUX-EN-AMIÉNOIS	SP98
Achicourt	E10
Achicourt	Gazole
Achicourt	SP98
Agde	E10
Agde	E85
Agde	Gazole
Agde	SP98
AGDE	E10
AGDE	E85
AGDE	Gazole
AGDE	GPLC
AGDE	SP95
AGDE	SP98
Agen	E10
Agen	E85
Agen	Gazole
Agen	SP98
AGEN	E10
AGEN	E85
AGEN	Gazole
AGEN	GPLC
AGEN	SP95
agneaux	E10
agneaux	Gazole
agneaux	SP98
Agneaux	E10
Agneaux	E85
Agneaux	Gazole
(67 rows)	

2- l'objectif ici était de voir au sein d'une même ville les différents prix entre station pour le Gasole. On y voit une certaine concurrence autour de 2,1 € et après il y a les extrêmes entre très proche de 2 € et proche de 3 €.

```
SELECT s.id_station, p.prix_carburant
FROM station s
JOIN prix p ON s.id_station = p.id_station
JOIN carburant c
ON p.id_carburant = c.id_carburant
WHERE s.nom_ville = 'PARIS' AND
c.nom_carburant = 'Gazole'
ORDER BY p.prix_carburant ASC
LIMIT 43;
```

```
saie_204_20200520-# SELECT S.I
urant = c.id_carburant WHERE
id_station | prix_carburant
```

id_station	prix_carburant
92300004	2.079
75008009	2.091
75013025	2.093
75020008	2.138
75017017	2.139
75013024	2.146
75012021	2.147
75018014	2.158
75016017	2.167
75013028	2.169
75016001	2.190
75015030	2.199
75017019	2.200
75016029	2.250
75008006	2.260
75010002	2.269
75015034	2.269
75015036	2.299
75016022	2.309
75016023	2.319
75016006	2.349
75007003	2.800
75001003	2.890

(23 rows)

3- ici nous voulions voir le top 7 des stations qui proposent un SP95 le moins cher parmi celles qui sont en ville, on a retiré celle des autoroutes. On remarque quelques villes d'Ile de France au top.

```
SELECT s.id_station, s.nom_ville,
p.prix_carburant
FROM station s
JOIN prix p ON s.id_station = p.id_station
JOIN carburant c
ON p.id_carburant = c.id_carburant
WHERE c.nom_carburant = 'SP95'
AND s.pop = 'R'
ORDER BY p.prix_carburant ASC
LIMIT 7;
```

id_station	nom_ville	prix_carburant
91080001	Évry-Courcouronnes	1.169
11620001	VILLEMOUSTAUSSOU	1.719
31800009	Estancarbon	1.724
95350002	SAINT-BRICE-SOUS-FORÊT	1.738
17330001	VILLENEUVE-LA-COMTESSE	1.800
93130007	Noisy-le-Sec	1.849
32500003	Fleurance	1.871

(7 rows)

4- ici c'est une requête très importante car ça nous permet de voir le nombre de rupture en fonction d'un carburant. Contrairement à ce que nous pouvons penser, le Gazole lui a le moins de rupture justement, car nous pensons qu'il est plus demandé donc il y a beaucoup plus de prévisions.

```

nom_carburant | nombre_rupture
-----+-----
GPLc          |          7531
SP95          |          6485
E85           |          5484
E10           |          3394
SP98          |          3218
Gazole        |           624
(6 rows)

```

```

SELECT c.nom_carburant, COUNT(r.id_station)
AS nombre_rupture
FROM rupture r
JOIN carburant c
ON r.id_carburant = c.id_carburant
GROUP BY c.nom_carburant
ORDER BY nombre_rupture DESC;

```

5- ici on peut remarquer que malgré des horaires de fermeture, des stations ont eu l'intelligence d'avoir des automates à 24 heures sur 24. Ils vendent même s'ils ne sont pas là.

```

SELECT s.nom_ville,
oh.heure_ouverture,
oh.heure_fermeture
FROM station s
JOIN ouverture_horaire oh ON
s.id_station = oh.id_station
WHERE oh.jour = 'Dimanche' AND
oh.automate = TRUE
LIMIT 34;

```

```

nom_ville | heure_ouverture | heure_fermeture
-----+-----+-----
Bourg-en-Bresse | 08:30:00 | 12:30:00
BOURG-EN-BRESSE | 08:30:00 | 12:30:00
Bourg-en-Bresse | 08:30:00 | 12:30:00
SAINT DENIS LES BOURG | 08:00:00 | 21:00:00
Bourg-en-Bresse | 08:30:00 | 12:30:00
Bourg en Bresse | 00:00:00 | 00:00:00
MONTCEAUX | 08:30:00 | 12:30:00
ARBENT - OYONNAX | 06:30:00 | 20:30:00
ARBENT | 08:30:00 | 12:30:00
MONTLUEL | 08:30:00 | 12:30:00
LA BOISSE | 08:30:00 | 12:30:00
La Boisse | 08:30:00 | 12:30:00
DAGNEUX | 08:30:00 | 12:30:00
LE POIZAT LALLEYRIAT | 08:30:00 | 12:30:00
NANTUA | 07:00:00 | 20:00:00
Saint-Didier-sur-Chalaronne | 09:00:00 | 12:30:00
LAGNIEU | 09:00:00 | 12:30:00
SAINT-VULBAS | 08:30:00 | 12:30:00
PONT-D'AIN | 08:30:00 | 12:30:00
NEUVILLE-SUR-AIN | 08:30:00 | 12:30:00
GEX | 08:30:00 | 12:30:00
GEX | 08:30:00 | 12:30:00
Saint-Bénigne | 08:30:00 | 12:30:00
Bellegarde-sur-Valserine | 08:30:00 | 12:30:00
FERNEY-VOLTAIRE | 08:30:00 | 12:30:00
SAINT-RAMBERT-EN-BUGEY | 08:30:00 | 12:30:00
Torcieu | 08:30:00 | 12:30:00
Ceyzériat | 08:30:00 | 12:30:00
JASSERON | 08:30:00 | 12:30:00
Champagne-en-Valromey | 08:30:00 | 12:30:00
COLIGNY | 08:30:00 | 12:30:00
LAIZ | 08:30:00 | 12:30:00
Chazey-Bons | 08:30:00 | 12:30:00
Confrançon | 08:30:00 | 12:30:00
(34 rows)

```

6- cette requête permet d'avoir vraiment beaucoup d'informations sur chacun des carburants. En effet, nous pouvons avoir leur prix minimum, leur prix maximum dans la France et également une moyenne des prix pour chacun des carburants. On a le SP98 et le gazole en haut et plutôt le E95 qui n'est pas aussi cher.

```
SELECT c.nom_carburant,  
COUNT(p.id_station) AS total_stations_vendent,  
MIN(p.prix_carburant) AS prix_minimum,  
MAX(p.prix_carburant) AS prix_maximum,  
AVG(p.prix_carburant) AS prix_moyen  
FROM prix p  
JOIN carburant c  
ON p.id_carburant = c.id_carburant  
GROUP BY c.nom_carburant  
ORDER BY prix_moyen DESC;
```

nom_carburant	total_stations_vendent	prix_minimum	prix_maximum	prix_moyen
SP98	6989	1.569	2.750	2.1635431392187724
Gazole	9389	1.250	2.903	2.1341048034934498
SP95	2946	1.169	2.620	2.1093170400543109
E10	6747	1.880	2.493	2.0673557136505113
GPLc	1444	0.600	2.020	1.0768691135734072
E85	3733	0.699	2.160	0.85090329493704795071
(6 rows)				